

ПРОГРАММИРОВАНИЕ

Введение в программирование

От алгоритма и первых языков — к Python, C# и C++ и базовым элементам синтаксиса.



Три понятия, с которых начинается программирование

В исходных материалах они образуют логическую цепочку: цель → язык → реализованная программа.

АЛГОРИТМ

01

Точное предписание исполнителю совершить последовательность действий для достижения цели за конечное число шагов.

ЯЗЫК ПРОГРАММИРОВАНИЯ

02

Формальный язык для записи алгоритмов; реализуется специальной программой — компилятором или интерпретатором.

ПРОГРАММА

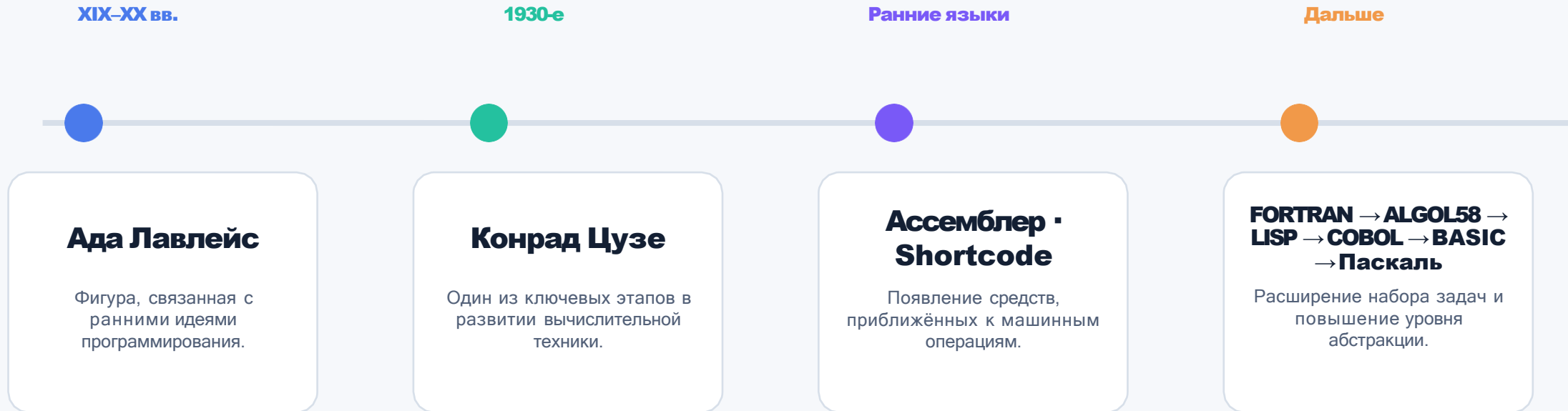
03

Последовательность машинных команд, предназначенная для достижения конкретного результата.

Главная идея: язык программирования — это способ записать алгоритм так, чтобы его можно было выполнить.

Как развивались языки программирования

Материал показывает переход от машинно-ориентированных средств к более абстрактным языкам.

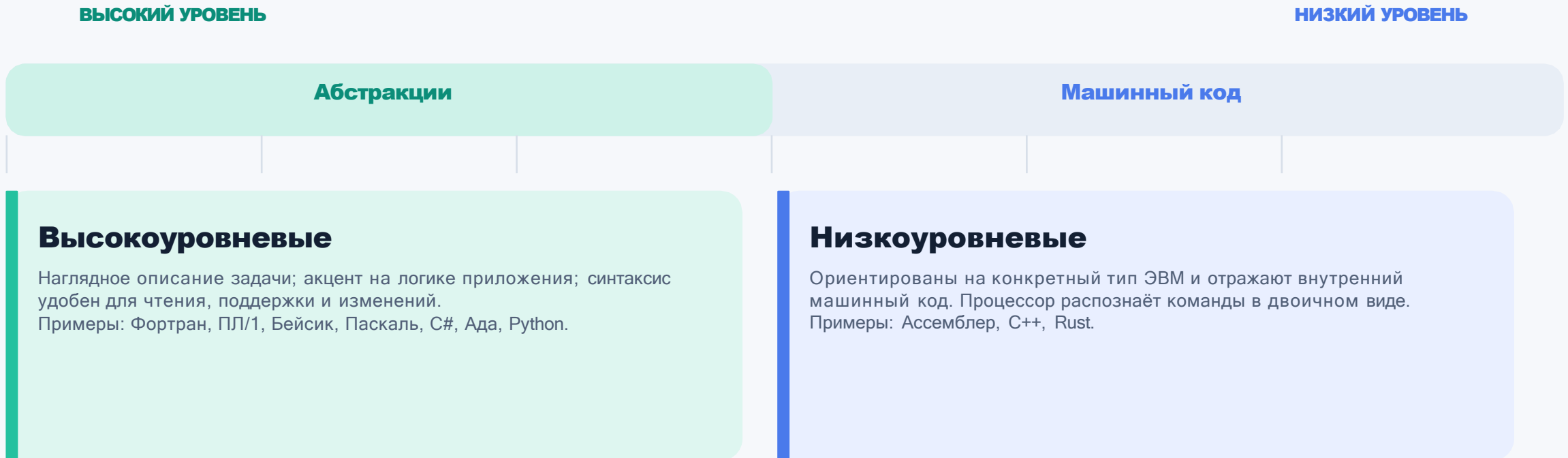


Линия развития

От команд, близких к машине, к языкам, в которых программист видит задачу на более понятном уровне.

Два уровня языка: ближе к человеку — ближе к машине

Исходный материал противопоставляет высокоуровневые и низкоуровневые языки по степени абстракции.



Главное различие — уровень абстракции и то, насколько близко запись находится к устройству компьютера.

Высокоуровневый и низкоуровневый языки — в одном кадре

Критерий	Высокоуровневые	Низкоуровневые	Примеры
Абстракция	Выше	Ниже	
Удобство чтения	Высокое	Ниже	
Связь с машиной	Опосредованная	Близкая	
Примеры	Python, C#, Ада	Ассемблер, C++, Rust	

Смысл классификации

Она помогает понять, где программист работает прежде всего с логикой задачи, а где — с устройством вычислений.

Python: простота синтаксиса и универсальность

В исходном материале Python представлен как высокоуровневый, читаемый и доступный начинающим язык.

PYTHON

Высокоуровневый

Интерпретируемый

Создан в конце 1980-х годов
Гвидо ван Россум

Простой синтаксис

Минималистичный и удобный для начинающих.

Динамическая типизация

Тип данных переменной определяется интерпретатором.

Большая библиотека

Поддержка веб-разработки, анализа данных, ИИ и автоматизации.

Удобная отладка

Интерпретация упрощает тестирование и поиск ошибок.

C#: объектно-ориентированная модель и экосистема .NET

Исходные материалы связывают язык с Microsoft, .NET и разработкой приложений на разных ОС.

ООП

Классы, объекты, наследование, полиморфизм.

Статическая типизация

Типы данных задаются явно, что помогает выявлять ошибки при компиляции.

Среда .NET

Выполнение программ в экосистеме .NET; кроссплатформенность через современный .NET.

Многозадачность

Удобные средства асинхронности, включая `async/await`.

2000

Год разработки языка
как части .NET Framework

C++: производительность и контроль над памятью

В исходнике C++ показан как язык, сочетающий высокоуровневые и низкоуровневые возможности.

C++

Расширение C с поддержкой
объектно-ориентированного
программирования

Высокая
производительность

Статическая типизация

ООП

Управление памятью

Контроль памяти

Низкоуровневые возможности позволяют работать с памятью и создавать производительные системы.

1

2

3

4

5

6

Память

1979

Год разработки Бьёрном
Страуструпом

Языки работают не изолированно

Исходный слайд о совместимости связывает языки с операционными системами, СУБД и другими языками.

Язык	ОС	СУБД	Связь с другими языками
C#	Windows → кроссплатформенность .NET	Интеграция с разными СУБД	C++, JavaScript/TypeScript, Python, Java, Ruby, PHP и др.
C++	Поддерживается основными ОС	Доступ через API и библиотеки	C
Python	Большинство популярных ОС	Множество СУБД через библиотеки и модули	C/C++, Java, C#, R

Вывод

Выбор языка связан не только с синтаксисом, но и с окружением, базами данных и средствами интеграции.

Синтаксис — правила, по которым «говорят» на языке

В исходном материале синтаксис включает грамматику, структуру и ключевые слова.

СИНТАКСИС =

Грамматика

Как строятся конструкции

Структура

Как организуется код

Ключевые слова

Специальные слова языка

Примеры из материала: `def`, `if`, `for` — ключевые слова Python.

Из чего складывается запись программы

Ключевые слова

def · if · for
Предопределённые слова со специальным значением.

Идентификаторы

Имена переменных, функций, классов и других сущностей.

Операторы

+ · - · * · /
and · or · not · == · != · < · >

Строки и комментарии

Отступы в Python; { } в C/Java.
Комментарии: # или // и др.

Один и тот же смысл может записываться по-разному: синтаксис зависит от конкретного языка.

Данные + условия + циклы = базовый каркас алгоритма

Эти элементы объединяют типы данных, структуры хранения и управляющие конструкции.

Типы данных

Простые: int, float, string, bool

Сложные структуры: массивы, списки, множества, словари и др.

Управление ходом выполнения

УСЛОВИЕ

ВЕТВЛЕНИЕ

ПОВТОРЕНИЕ

if / else / elseif / switch

for / while

Условия принимают решения, циклы повторяют действия.

Что стоит запомнить

01

Алгоритм

последовательность действий для достижения цели

02

Язык

формальный способ записать алгоритм

03

Уровень

высокий = больше абстракции; низкий = ближе к машине

04

Синтаксис

правила, структура и ключевые элементы записи

05

Языки

Python, C# и C++ решают разные задачи и работают в разных экосистемах

Программирование — это путь от идеи и алгоритма к понятной записи, которую компьютер может выполнить.